

Retrieval-Grounded MCQ Generation for a Generative-AI Course: A Multi-Agent Approach with Fine-Tuning, Critique, and Feedback

1. Project Goal and Scenario

This project developed an AI-powered study and assessment assistant designed for a university course on Generative AI and Large Language Models. The assistant helps instructors create multiple-choice questions (MCQs) directly from lecture content, with adjustable difficulty levels and alignment to Bloom's Taxonomy learning objectives. It also generates realistic distractors that challenge common misunderstandings rather than relying on obvious wrong answers. For students, the system provides automatic grading, detailed answer explanations, and references to the specific lecture slides where the concepts were taught. By transforming static lecture materials into an interactive learning experience, the assistant serves as both a personalized tutor and a reliable practice-exam platform.

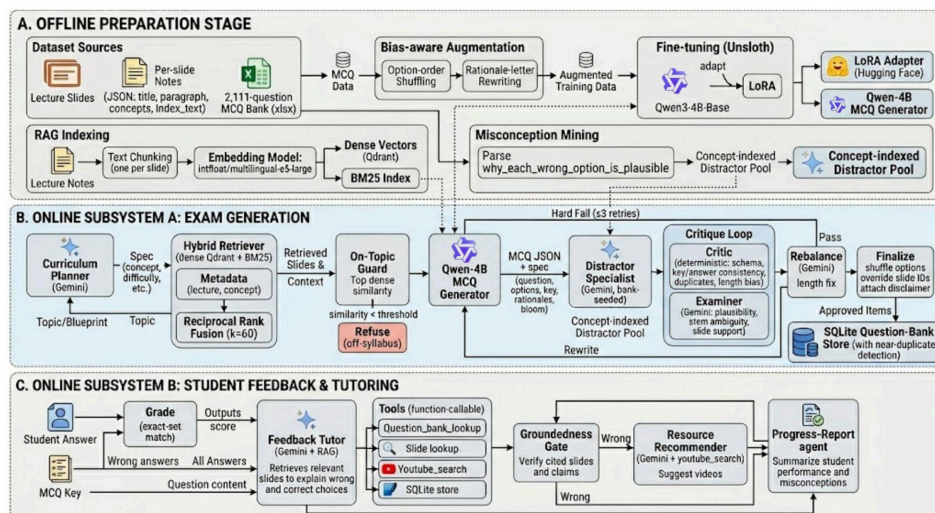
Target users

- Instructors and teaching assistants, who need to produce balanced, course-aligned exams quickly without hand-writing every item.
- Students who want unlimited practice questions grounded in what they were actually taught, plus explanations and follow-up resources when they answer incorrectly.

Typical tasks and value

- "Generate a balanced 10-question exam on the Transformer and RAG lectures."
- "Give me an Intermediate, Analyze-level question on RLHF with four options."

2. System Overview and Technology Stack



The system is organized into three stages. (1) An offline preparation stage builds the training and retrieval resources: it constructs the MCQ dataset, applies bias-aware augmentation, fine-tunes the generator with LoRA, indexes the lecture notes for hybrid retrieval, and mines a concept-indexed distractor pool. (2) An

online exam-generation subsystem chains a curriculum planner, a hybrid retriever, an on-topic guard, a fine-tuned generator, a distractor specialist, a critique loop, and a finalization step into a LangGraph state machine. (3) An online student-feedback subsystem grades answers, produces grounded explanations, recommends resources, and generates progress reports. The full architecture is shown in Figure 1 of the accompanying paper.

Models, APIs, frameworks, and tools used

Component	Choice
Generator model	Qwen3-4B-Base, fine-tuned with LoRA (4-bit QLoRA)
Reasoning agents (LLM)	Google Gemini (gemini-3.1-flash-lite) via google-genai
Embedding model	intfloat/multilingual-e5-large (prototype: Qwen3-Embedding-0.6B)
Vector database	Qdrant (dense, cosine)
Sparse retrieval	BM25 (Okapi)
Fine-tuning stack	Unsloth + TRL/PEFT (QLoRA)
Orchestration	LangGraph (StateGraph) / LangChain
Tools	question_bank_lookup, slide_lookup, youtube_search
Persistent storage	SQLite (question bank + interaction records)
Model hosting	Hugging Face Hub (LoRA adapter)
Evaluation judge	GPT-5.5 Thinking (independent LLM-as-a-judge)

Table 1. Models, APIs, frameworks, and tools used in the assistant.

3. Dataset

We constructed a lecture-grounded MCQ dataset of 2,111 questions from nine course lectures. Three sources were combined: the course lecture slides, structured per-slide notes generated in JSON (capturing concepts, definitions, mechanisms, examples, and slide-level evidence); and an MCQ exam bank assembled from lecture-derived questions plus exam-style items inspired by external educational resources (NotebookLM and DeepLearning.AI material). Introductory, outline, divider, and reference-only slides were excluded, so the dataset focuses on examinable content. Every item carries the question text, type, options, correct answer, difficulty, Bloom level, tested concepts, evidence slide IDs, and rationales for the correct and incorrect options.

Category	Breakdown	Count
Total	All MCQ items	2,111
Question type	Single-answer / True-False / Multi-response	1,656 / 276 / 179
Difficulty	Beginner / Intermediate / Difficult	664 / 887 / 560
Bloom level	Remember / Understand / Apply	579 / 491 / 291
Bloom level	Analyze / Evaluate / Create	442 / 196 / 112

Category	Breakdown	Count
Lecture coverage	L1–L9 (per lecture)	178–298

Table 2. Summary statistics of the constructed MCQ dataset (2,111 items, nine lectures).

4. Prompt Design

Every agent is driven by a role-constrained system prompt that fixes its role, style, and output contract, and the reasoning agents are pinned to JSON output. Three principles recur across all prompts: grounding (use only the provided lecture content), structured output (a fixed JSON schema rather than free text), and explicit constraints on allowed values (difficulty, Bloom level, question type). The MCQ-generator prompt is also the training-time system prompt, so the fine-tuned model and the prompted agents share a single contract.

MCQ Generator (system prompt). *"You are an exam-question writer for a university course on Generative AI and Large Language Models. Given lecture content and a generation specification, produce exactly one multiple-choice question in valid JSON format. Use ONLY information present in the lecture content; do not invent facts. Match the requested question_type exactly (single_answer: four options A–D, one correct; true_false; multi_answer). Wrong options must target real misconceptions. Cite slides in evidence_slide_ids as L{lecture}-S{slide}. Output ONLY the JSON object."*

The remaining agents specialise in this pattern. The Curriculum Planner converts a topic and target difficulty into a JSON spec (concept, difficulty, bloom_level, question_type, retrieval_query) constrained to the allowed enumerations, and an Exam Blueprint planner composes a balanced set of such specs so no Bloom level, difficulty, or type dominates. The Distractor Specialist rewrites wrong options into plausible, misconception-based distractors explicitly required to be similar in length and specificity to the correct answer—directly countering length bias. The Feedback Tutor explains a student's answer using only retrieved slide evidence and must cite slide IDs, and the Resource Recommender is instructed to call the youtube_search tool rather than invent links. The Examiner is a strict pass/fail quality reviewer that judges distractor plausibility, stem ambiguity, and whether the cited slides actually support the marked answer.

5. Retrieval-Augmented Generation (RAG)

The retrieval corpus is built from the slide-level notes of the nine lectures. Each slide is treated as one retrieval chunk, combining the slide title, explanation, and key concepts into a single passage, with metadata for lecture number, slide number, title, and concepts. This yields 664 lecture-based chunks, and the same slide identifiers serve as the evidence citations attached to generated MCQs and to student feedback.

Retrieval is hybrid. Dense embeddings are stored in a local Qdrant database with cosine similarity, while a BM25 index over the same chunks captures exact keyword matches. The production embedding model is multilingual-e5-large (normalised 1024-dimensional vectors, query:/passage: format). At query time, both retrievers return candidates that are merged with Reciprocal Rank Fusion (equal weights), and the top 4–5 chunks are passed to the generator as grounding evidence. Retrieval can be filtered by lecture or concept, so the planner can target specific parts of the course.

Two guards protect quality. An on-topic guard checks the top dense similarity and rejects the request—routing it to a refusal node—when the score falls below 0.78, preventing off-syllabus generation. A near-duplicate check (cosine ≥ 0.93) prevents almost-identical questions from entering the bank. In the final

system, RAG grounds MCQ generation, assigns reliable evidence slide IDs (the retriever's IDs override anything the model produced), and authors lecture-based feedback when students answer incorrectly.

6. Fine-Tuning / Parameter-Efficient Tuning (PEFT)

To adapt a general model to the MCQ format, we fine-tuned Qwen3-4B-Base with Quantized Low-Rank Adaptation (QLoRA) using the Unsloth framework. A base model was chosen deliberately, to keep full control of the JSON output structure and avoid the instruction variant's reasoning traces. The model is loaded in 4-bit precision, and rank-16 LoRA adapters (alpha 32, dropout 0) are attached to all seven attention and MLP projections, so only a small fraction of parameters are trained. Training uses three epochs, a 1e-4 cosine-decayed learning rate, an effective batch size of 16, a response-only loss (masked to the assistant turn), and early stopping on validation loss; the data is split 70/15/15 stratified by difficulty and Bloom level.

Because human-authored banks cluster the correct answer at particular positions, a bias-aware augmentation step is applied before training: option order is shuffled so the correct answer rotates uniformly across positions, and the letter references embedded inside the rationales are rewritten to match the new layout, keeping every field consistent. The resulting LoRA adapter is published to the Hugging Face Hub and reloaded at serving time.

Behaviour change vs. the base model

On a held-out test set of 60 items, the effect of fine-tuning is decisive. The base model produces readable text but essentially never the required MCQ structure—0% output completeness, 0% answer-key consistency, and 0% usable rationales—so it cannot author a valid item. The fine-tuned model reaches near-perfect structure, raising the overall score from 6.75 to 90.75 (Table 3, Figure 1).

Metric	Base	Fine-tuned
Output completeness	0.00	1.00
Option-count correctness	0.27	0.98
Answer-key consistency	0.00	0.90
Rationale quality	0.00	0.75
Overall score (0–100)	6.75	90.75

Table 3. Base vs. fine-tuned generator on the held-out test set ($n = 60$).

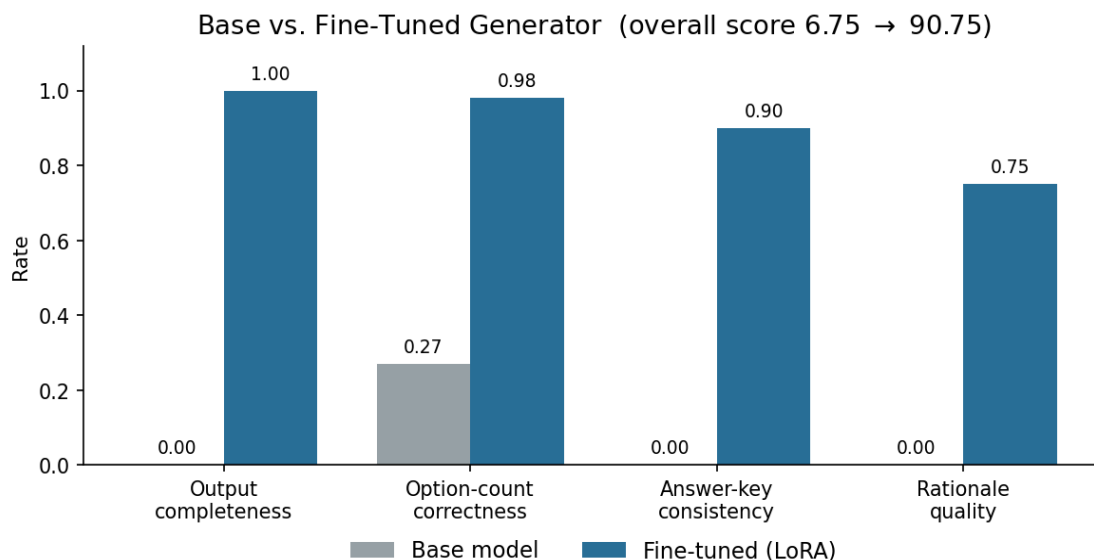


Figure 1. Structural quality of the base vs. fine-tuned generator. LoRA makes the generation task feasible.

7. Tools and Function Calling

The system integrates several tools that specific agents may call, which keep generated questions, explanations, and recommendations traceable rather than invented.

- **question_bank_lookup** — checks whether a newly generated MCQ is too similar to a previously approved one (cosine ≥ 0.93) before storing it, reducing near-duplicates and improving exam diversity.
- **slide_lookup** — retrieves the lecture evidence associated with a question, so both the generator and the Feedback Tutor can cite the exact source slides.
- **youtube_search** — called by the Resource Recommender via function calling to suggest real external videos for a weak concept, instead of hallucinating links.
- **SQLite store** — persists the approved question bank and student-interaction records, enabling deduplication, history, and progress reporting.

When a student answers incorrectly, the Resource Recommender's `youtube_search` call meaningfully improves the result: rather than producing a plausible-looking but possibly fake link, the tool returns a genuine, retrievable video for the missed concept—turning a wrong answer into a grounded next step.

8. Multi-Agent Design

Rather than relying on one model for planning, retrieval, generation, validation, and feedback, the workflow decomposes the task into specialized agents that collaborate through a shared LangGraph state. The generation graph follows the sequence: Planner → Retriever → Guard → Generator → Distractor Specialist → Critic (with Examiner) → Rebalance → Finalize, with conditional routing that loops a failing item back to the Generator until it passes or a retry limit (≤ 3) is reached.

- **Curriculum Planner / Blueprint (Gemini)** — sets the concept, difficulty, Bloom level, type, and retrieval query, and composes a balanced exam blueprint.

- **Retriever + On-Topic Guard** — hybrid RAG supplies grounding evidence; the guard refuses off-syllabus requests below the similarity threshold.
- **MCQ Generator (fine-tuned Qwen3-4B)** — produces the structured MCQ JSON constrained to the retrieved evidence.
- **Distractor Specialist (Gemini)** — rewrites weak, wrong options into plausible, misconception-based distractors of comparable length.
- **Critic (deterministic) + Examiner (Gemini)** — the Critic checks schema, required fields, type consistency, answer-key validity, duplicates, and length bias; the Examiner judges plausibility, ambiguity, and slide support. Either failing returns the item for regeneration.
- **Rebalance + Finalize** — a last length-bias fix, then option-position shuffling, citation override with the retriever's slide IDs, and the disclaimer, before storage.

For a question on PPO entropy loss, the Generator's first draft contains an implausible distractor and makes the correct option the longest. The Critic flags the length bias, and the Examiner marks `distractors_plausible = false`, so the item is routed back; the Distractor Specialist rewrites the wrong options from the concept-indexed misconception pool, the Rebalance step equalises option lengths, and Finalize shuffles positions and attaches the cited slides. The result is a balanced, grounded item that no single agent produced alone.

9. Evaluation Method and Key Findings

Good performance for this assistant means structurally valid questions, grounded in the lectures, pedagogically useful, and free of exploitable artifacts (position/length bias, weak distractors). We evaluate two adaptation contributions separately, giving two baseline comparisons: base vs. fine-tuned (the effect of LoRA) and single- vs. multi-agent (the effect of the orchestration). We report automatic structural metrics from a deterministic scorer and quality metrics from an independent LLM-as-a-judge (GPT-5.5 Thinking) rating eight criteria on a 0–10 scale; using a judge from a different model family than the system's own Gemini agents reduces self-preference bias.

Single- vs. multi-agent: automatic metrics

On 100 generated questions, the multi-agent graph drives grounding to a perfect 1.00 (from 0.92) and cuts the length-bias rate from 0.618 to 0.000; the duplicate rate is already negligible in both. The length rate drops below the 0.25 ideal, a one-sided over-correction we note as a tuning target rather than a defect.

Metric	Single-agent	Multi-agent
Duplicate-option rate (lower better)	0.01	0.01
Longest-is-correct (ideal ≈ 0.25)	0.618	0.000
Grounded item rate (higher better)	0.92	1.00

Table 4. Single- vs. multi-agent automatic metrics ($n = 100$; length scored on the 34 single-answer items).

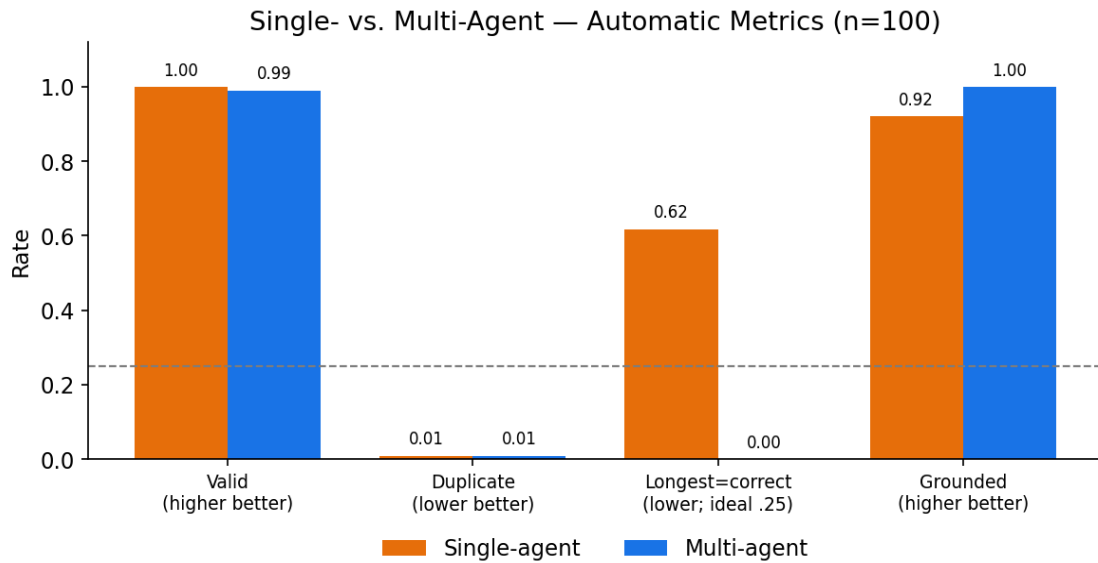


Figure 2. Automatic structural metrics. The graph removes length bias and achieves perfect grounding.

Single- vs. multi-agent: LLM-as-a-judge

Both configurations score about 9/10 overall because the fine-tuned generator already produces strong items, so the orchestration's value concentrates where the generator is weakest. The multi-agent system improves exactly the dimensions it targets—distractor quality (6.96 → 7.36), lecture grounding (9.68 → 9.84), and workflow consistency (9.65 → 9.81)—at a small cost to a few others, for a marginally higher overall (8.97 → 9.00). In other words, the orchestration reallocates quality toward the properties that most determine assessment validity.

Criterion	Single-agent	Multi-agent
Question-type compliance	9.96	9.90
Answer-key correctness	9.95	9.78
Lecture grounding/factuality	9.68	9.84
Concept alignment	9.05	8.97
Difficulty alignment	7.84	7.74
Distractor quality	6.96	7.36
Clarity / ambiguity	8.65	8.57
Workflow consistency	9.65	9.81
Overall	8.97	9.00

Table 5. LLM-as-a-judge quality scores (0–10), rated by GPT-5.5 Thinking.

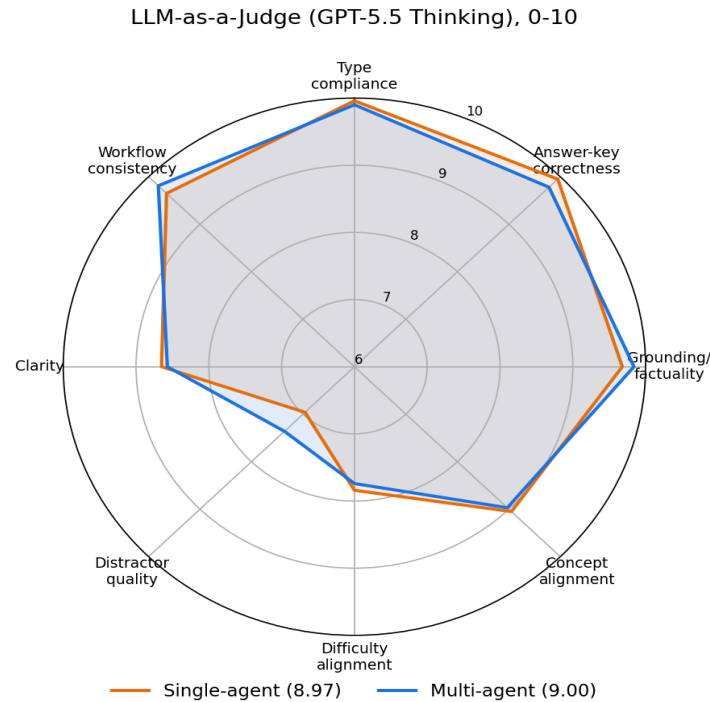


Figure 3. Judge profile across eight criteria: the multi-agent system expands on grounding, distractor quality, and workflow consistency.

Fine-tuning is essential—it lifts the overall generation score from 6.75 to 90.75 and is what makes the task feasible at all. The multi-agent layer then improves grounding (0.92 → 1.00), removes answer-length bias (0.618 → 0.000), and raises distractor quality (6.96 → 7.36), the central research problem of MCQ generation.

10. Ethics, Safety, and Limitations

Visible disclaimer

Every generated question and explanation carries a visible disclaimer: *"This question and explanation are AI-generated from course lecture notes and may contain errors. Always verify against the original slides and your instructor."*

Rules for refusal and caution

The assistant is deliberately domain-restricted to the nine-course lectures. An on-topic guard refuses any request whose retrieved evidence falls below a similarity threshold (0.78), routing it to a refusal node rather than generating an unsupported question. The Examiner additionally fails any item whose marked answer is not supported by the cited slides or whose stem is ambiguous, sending it back for regeneration or dropping it. The assistant does not offer open-domain advice and declines requests outside the course scope.

Risks and mitigations

- Hallucination / weak grounding** — RAG retrieves lecture evidence before generation, the retriever's slide IDs override any the model invented, and a groundedness check keeps both questions and feedback tied to the course content.

- **Bias (position and length)** — balanced training augmentation plus inference-time option shuffling addresses answer-position bias; a Critic length check and a distractor-rebalancing step address length bias.
- **Misleading distractors** — the Distractor Specialist and Examiner ensure wrong options are plausible misconceptions rather than obviously incorrect or unrelated.
- **Harmful or out-of-scope advice** — the domain restriction and refusal node prevent the assistant from answering beyond the vetted course material.
- **Privacy** — no sensitive or personal student data is used; the corpus is course lecture material, and external sources (NotebookLM, DeepLearning.AI) were used only as inspiration. Student interactions are stored locally in SQLite and aggregated for progress reports without personally identifying information.
- **Over-reliance / academic integrity** — the disclaimer and the recommendation to verify against the instructor position the tool as a study aid, not an authoritative exam source; an instructor-approval step before questions are served to students is recommended for deployment.

Limitations and future improvements

- The LLM-as-a-judge uses a single automatic rater; absolute scores should be read comparatively, and a human-rating study with instructors or TAs would strengthen validation.
- The length-bias rebalancing overcorrects (to 0.000); it should be tuned toward the unbiased 0.25 for four-option items.
- Difficulty alignment is the weakest dimension for both systems (~7.8); calibrating true reasoning difficulty remains open.
- Future work: stronger human validation, improved difficulty calibration, an instructor-approval gate, and deeper integration of student-performance data into adaptive tutoring.

11. Conclusion

This report presented a retrieval-grounded, multi-agent assistant for generating and explaining MCQs in a Generative-AI course. It combines a fine-tuned Qwen3-4B generator, a hybrid RAG pipeline grounded in lecture slides, and a LangGraph multi-agent workflow for planning, generation, critique, distractor refinement, rebalancing, and student feedback. Fine-tuning teaches the model the required MCQ structure (6.75 → 90.75), RAG keeps the output tied to the lecture material, and the multi-agent workflow improves grounding, removes length bias, and strengthens distractors before a question is finalized. Together with the tutoring component, the system turns a static MCQ bank into an adaptive, trustworthy learning environment, while explicit guards, disclaimers, and bias mitigations keep it responsible by design.